

Task 18: Missing Values in Pandas

Track: Data Science | Level 1 - Day 18 | Tools: Python, Pandas, Jupyter Notebook

Level 1 - Day 18: Missing Values in Pandas

Track: Data Science

Task: Task 18 - Missing Value Auditing, Detection, and Imputation

Objective: Identify missing values in tabular datasets using `isna()` and `notna()`, generate missing-value statistical summaries, compare dropping vs. filling strategies, and produce fully cleaned datasets.

1. Quick Reference: Missing Value Handling Techniques in Pandas

Operation / Function	Syntax Example	When to Use
Detection	<code>df.isna()</code> , <code>df.notna()</code>	Identifying null/NaN elements and creating boolean filter masks
Summary / Audit	<code>df.isna().sum()</code> , <code>df.isna().mean() * 100</code>	Quantifying missing volume and percentages per feature column
Row Removal	<code>df.dropna(subset=['critical_col'])</code>	When rows lack primary identifiers or target labels
Column Dropping	<code>df.dropna(axis=1, thresh=len(df)*0.6)</code>	When missingness exceeds high thresholds (>40-50%)
Central Tendency Imputation	<code>df['col'].fillna(df['col'].mean())</code> / <code>df['col'].median()</code>	Numerical columns with low skew (mean) or high skew/outliers (median)
Categorical Imputation	<code>df['col'].fillna(df['col'].mode()[0])</code> / <code>'Unknown'</code>	Missing categorical values or survey responses
Interpolation / Forward Fill	<code>df.ffill()</code> , <code>df.bfill()</code> , <code>df.interpolate()</code>	Time-series, sequential telemetry, and financial data

2. Setting Up the Raw Dataset with Realistic Missingness

In [code]

```
import pandas as pd
import numpy as np

# Create an uncleaned student and employee performance dataset with synthetic null values
raw_data = {
    'StudentID': ['S101', 'S102', 'S103', 'S104', 'S105', 'S106', 'S107', 'S108'],
    'Name': ['Aman', 'Pooja', 'Rohan', 'Sneha', 'Vikas', 'Aditi', 'Karan', 'Neha'],
    'Gender': ['Male', 'Female', np.nan, 'Female', 'Male', 'Female', np.nan, 'Female'],
    'Age': [20, 22, np.nan, 21, 23, np.nan, 22, 21],
    'Math_Score': [85.0, 92.0, 78.0, np.nan, 65.0, 98.0, np.nan, 74.0],
    'Reading_Score': [88.0, np.nan, 72.0, 90.0, 60.0, 95.0, 58.0, 80.0],
    'Attendance_Pct': [92.5, 88.0, np.nan, 95.0, 70.0, 99.0, 65.0, np.nan],
    'Fee_Status': ['Paid', 'Pending', 'Paid', 'Paid', np.nan, 'Paid', 'Pending', 'Paid']
}

df = pd.DataFrame(raw_data)
print("=== RAW DATASET SNAPSHOT ===")
df
```

3. Missing-Value Auditing & Statistical Summary

In [code]

```
# Generate a comprehensive missing-value audit report
missing_counts = df.isna().sum()
missing_pct = (df.isna().mean() * 100).round(2)

missing_summary = pd.DataFrame({
    'Missing_Count': missing_counts,
    'Missing_Percentage (%)': missing_pct,
    'Dtype': df.dtypes
})

print("=== MISSING VALUE AUDIT SUMMARY ===")
missing_summary
```

4. Detection & Filtering using `isna()` and `notna()`

In [code]

```
# Example 1: Isolating rows that have ANY missing score in Math
missing_math = df[df['Math_Score'].isna()]
print("Rows with Missing Math_Score:")
print(missing_math[['StudentID', 'Name', 'Math_Score']])

# Example 2: Filtering complete records using notna()
valid_age_attendance = df[df['Age'].notna() & df['Attendance_Pct'].notna()]
print(f"\nTotal records with valid Age AND Attendance: {len(valid_age_attendance)} out of {len(df)}")

# Example 3: Rows with missing Gender
unspecified_gender = df[df['Gender'].isna()]
print(f"Records missing gender: {list(unspecified_gender['StudentID'])}")
```

5. Comparative Evaluation: Dropping vs. Imputation Strategies

In [code]

```
# Strategy A: Listwise Row Deletion (dropna)
df_dropped = df.dropna()
print(f"Original rows: {len(df)} | Rows after dropping all nulls: {len(df_dropped)}")
print(f>Data loss percentage: {((len(df) - len(df_dropped)) / len(df)) * 100:.1f}%")

# Notice that dropping all rows causes severe information loss!
```

6. End-to-End Cleaning & Preprocessing Pipeline

Applying disciplined, column-specific imputation rules:

1. ``Age``: Impute with **Median** (robust against demographic skew).
2. ``Math_Score`` & ``Reading_Score``: Impute with **Mean** rounded to 1 decimal place.
3. ``Attendance_Pct``: Impute with **Median** percentage.
4. ``Gender`` & ``Fee_Status``: Impute with **Mode** (most frequent class) or designated category.

In [code]

```
# Create clean copy
df_cleaned = df.copy()

# 1. Numerical Imputations
age_median = df_cleaned['Age'].median()
math_mean = round(df_cleaned['Math_Score'].mean(), 1)
reading_mean = round(df_cleaned['Reading_Score'].mean(), 1)
attendance_median = round(df_cleaned['Attendance_Pct'].median(), 1)

df_cleaned['Age'] = df_cleaned['Age'].fillna(age_median)
df_cleaned['Math_Score'] = df_cleaned['Math_Score'].fillna(math_mean)
df_cleaned['Reading_Score'] = df_cleaned['Reading_Score'].fillna(reading_mean)
df_cleaned['Attendance_Pct'] = df_cleaned['Attendance_Pct'].fillna(attendance_median)

# 2. Categorical Imputations
gender_mode = df_cleaned['Gender'].mode()[0]
fee_mode = df_cleaned['Fee_Status'].mode()[0]

df_cleaned['Gender'] = df_cleaned['Gender'].fillna(gender_mode)
df_cleaned['Fee_Status'] = df_cleaned['Fee_Status'].fillna(fee_mode)

# 3. Final Verification
print("=== CLEANED DATASET SNAPSHOT ===")
print(df_cleaned)

remaining_nulls = df_cleaned.isna().sum().sum()
print(f"\nTotal remaining null values across dataset: {remaining_nulls}")
```

7. Downstream Analytics on Cleaned Dataset

In [code]

```
# Compute Composite Total & Average Score on Cleaned Data
df_cleaned['Composite_Average'] = ((df_cleaned['Math_Score'] + df_cleaned['Reading_Score']) /
2).round(2)
df_cleaned['Pass_Status'] = np.where(
    (df_cleaned['Composite_Average'] >= 75) & (df_cleaned['Attendance_Pct'] >= 75),
    'High Distinction',
    'Regular Pass'
)

print("=== FINAL PREPROCESSED DATASET WITH DERIVED METRICS ===")
df_cleaned[['StudentID', 'Name', 'Math_Score', 'Reading_Score', 'Composite_Average', 'Attendance_Pct',
'Pass_Status']]
```